COMP2040 – Operating Systems

Lab 05: Address Translation and Memory Allocation

Overview

This lab covers Lecture 7. In Tasks 1 to 3 you complete `translate.c`, a small program that translates virtual addresses the way the MMU does: split the address, look in the TLB, and on a miss read the page table. In Task 4 you write a program yourself, with no starter code. Task 5 is on paper. Build the starter once, and again after each change:

```
gcc -std=c11 -Wall -Wextra translate.c -o translate
```

Submission

Upload **one file** to Canvas before the deadline shown there: `StudentID_Lab05.zip`, containing your report `StudentID_Lab05.pdf`, your completed `translate.c` and your `fit.c`. In the report, for each task, give the commands, the relevant output (as text) and short answers to the questions.

1 Split the address

The address space has 16 pages of 256 bytes. Complete `page_number()` and `page_offset()` (`TODO 1`), then run `./translate checks`. Every access still says `invalid address` until Task 2; for now, check the page and offset columns.

Include in your report.

(a) The page and offset of each address in the output.
(b) Find the page and offset of `0x3A2` by hand. Why does the last hex digit pair give the offset directly?
(c) A 1000-byte region is placed in 256-byte pages. How many pages does it use, how many bytes are unused, and what kind of fragmentation is this?

2 Read the page table

Each entry of `page_table` says whether the page is mapped, whether it is in memory, and whether writes are allowed. Complete `check_page_table()` (`TODO 2`) so that it returns `INVALID` if the page is outside the table or not mapped, `NOT_RESIDENT` if it is mapped but not in memory, and `OK` otherwise. Now `R 519` must give physical address 263.

Include in your report.

(a) The output of `./translate checks`.
(b) Explain the result of `W 5`, `R 1300` and `R 1536`. Which of the three are errors in the program, and which one is not?
(c) Why must your check for page 16 come before you read `page_table[page]`?

3 Add the TLB

Complete `tlb_lookup()` (`TODO 3`): search the filled entries for the page, count a hit and return the entry, or count a miss and return `NULL`. Then `./translate checks` must report 2 hits and 6 misses. Then run `./translate local`, which reads pages 2, 3 and 4 in order, and `./translate scattered`, which reads 11 pages in no useful order.

Include in your report.

(a) Both hit rates, and why they are so different.
(b) The effective access time of each workload, using your hit rates with $t = 2\,\text{ns}$ and $m = 100\,\text{ns}$ (the model on slide 56).
(c) Change `TLB_SIZE` to 8 and run both again. What changes, what does not, and why?

4 Write your own program: fit.c

Write `fit.c` yourself; there is no starter code. It simulates first-fit, best-fit and worst-fit on a list of free holes and a list of requests. We give only the input files `fit_lecture.txt` and `fit_lab.txt`. Each contains the number of holes, the hole sizes in address order, the number of requests and the request sizes. Read them with `scanf()`, compile with the same flags as before, and run the program as `./fit < fit_lab.txt`.

Serve the requests in order; no process exits. A request takes its space from the chosen hole and what is left stays a free hole. If two holes tie, the earlier one wins. **Every policy starts again from the original holes.** Print one block per policy, with one line per request. For `fit_lecture.txt`, your results must match slide 28:

```
First-fit:
P1 212 -> H2
...
P4 426 -> not allocated
Holes left: 100 176 200 300 183
Total free: 959  Largest hole: 300
```

Include in your report.

(a) The output for `fit_lab.txt`. You do not need to explain your code.
(b) Under first-fit, the 400 request is not allocated. How much memory was free at that moment, and what is this problem called?
(c) Best-fit places everything here. Does that make it the best policy? Why is the space left in a hole not internal fragmentation?

5 On paper

A machine has 32-bit virtual addresses, 2^{22} bytes of physical memory and 4 KiB pages. A process uses the virtual address `0x11123456`.

(a) How many pages and how many frames are there?
(b) Give the page number and offset in hex. With a two-level table (10, 10 and 12 bits), give the outer and inner index.
(c) With 4-byte entries, how large is a flat page table? How much does a two-level table need if only three 4 MiB-aligned regions are in use?
(d) Which steps of this translation does the hardware do, and what does the OS do?